

Automated Log Archival & Monitoring Pipeline on AWS

A secure, low-overhead operations framework featuring automated Ubuntu EC2 local log and database backup serialization, granular S3 storage, and proactive CloudWatch telemetry alerts.

Project Overview & Objectives

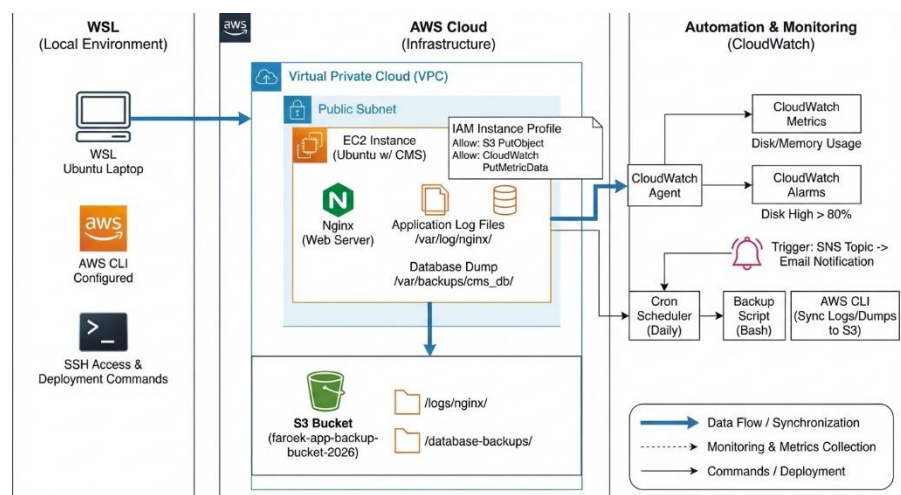
This project establishes an automated, highly resilient cloud operations pipeline designed to host, protect, and monitor a self-hosted CMS and database application on AWS. The primary engineering goal is to completely eliminate manual log management and disaster recovery overhead by establishing secure, automated local backup routines on an Ubuntu EC2 instance, transmitting them into private Amazon S3 object storage, and integrating system-wide metrics and notifications through Amazon CloudWatch.

Core Technical Objectives Met:

- **End-to-End Automation:** Daily cron schedule automatically captures web server log events and serialized MySQL/MariaDB database states.
- **Least-Privilege Security Design:** Implemented explicit IAM policies restricting the compute nodes to exact API endpoints (PutObject) required for backups.
- **Proactive Performance Alerting:** The system monitors disk storage and CPU limits, leveraging CloudWatch Alarms and AWS SNS to email alerts instantly upon threshold breaches.

System Architecture & Data Flow

The visual blueprint below details the complete pipeline workflow, illustrating the secure integration between local development platforms, cloud computing virtual environments, durable storage assets, and central cloud telemetry pipelines:



Tech Stack & Environment

Infrastructure Component	Category	Operational Role & Configuration
WSL Ubuntu (Local Windows)	Local Development	Operational deployment terminal with AWS CLI installed for cloud administration and SSH management
AWS EC2 (Ubuntu 22.04)	Compute Layer	Hardened VM hosting Nextcloud application stack, local cron processes and CloudWatch agent
Amazon S3	Storage Layer	Encrypted private backup store (<i>farook-backup-bucket-2026</i>) using partitioned folder structure
Amazon Cloudwatch	Telemetry & Alerts	Central monitoring agent pulling memory and disk usage metrics, driving administrative alert boundaries
AWS SNS	Messaging Pipeline	Simple notification service topic dispatching metrics alarm boundaries <u>directly</u> to administrator emails.
Nginx & Maria DB	Workload Stack	Secure LEMP web server and transactional database engine hosting the self-hosted CMS layer

Step-by-Step Implementation Guide

1. Local Environment & SSH Key Management

To establish secure operational management, we configure WSL Ubuntu to manage the remote cloud server and initialize the AWS CLI environment. The SSH private key file permissions must be tightly restricted to prevent local unauthorized access:

```
# Secure local SSH private key permissions
chmod 400 ~/.ssh/my-aws-key.pem

# Configure AWS Command Line Interface for remote bucket management
aws configure
# AWS Access Key ID [***]: <YOUR_ACCESS_KEY>
# AWS Secret Access Key [***]: <YOUR_SECRET_KEY>
# Default region name [us-east-1]: us-east-1
# Default output format [json]: json
```

2. Secure S3 Bucket & IAM Security Provisioning

A private S3 bucket is created to store off-site archives. To adhere to cloud security standard frameworks, we enforce the Principle of Least Privilege by designing a custom IAM policy and role. The policy permits the EC2 instance ONLY to write objects to the designated backup bucket, blocking all other actions :

DataNestEC2BackupAndMonitoringPolicy

EC2 instance needs permissions to upload backups to S3 and report custom disk memory metrics to CloudWatch.

EditDelete

Policy details

TypeCustomer managed

Creation timeSeptember 05, 2026, 15:01 (UTC+08:00)

Edited timeSeptember 05, 2026, 15:01 (UTC+08:00)

ARNarn:aws:iam::372303521665:policy/DataNestEC2BackupAndMonitoringPolicy

Permissions

Entities attached

Tags

Policy versions (1)

Last Accessed

Permissions defined in this policy

Permissions defined in this policy document specify which actions are allowed or denied. To define permissions for an IAM identity (user, user group, or role), attach a policy to it (3)

Search

Allow (3 of 475 services)

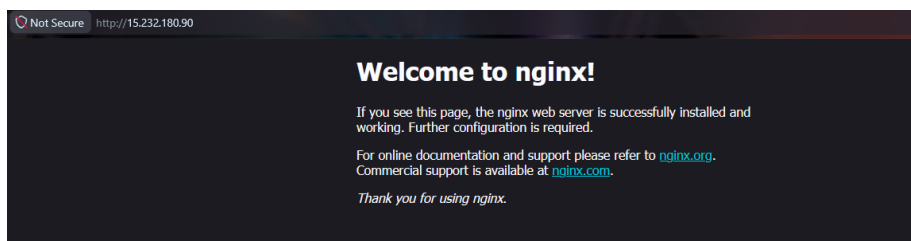
Show remaining 472 services

Service	Access level	Resource	Request condition
CloudWatch	Limited: Write	All resources	None
CloudWatch Logs	Limited: List, Write	All resources	None
S3	Limited: List, Read, Write	Multiple	None

3. Ubuntu Web Stack & CMS Application Deployment

An EC2 virtual server (Ubuntu LTS) is provisioned within a public subnet, attaching the custom IAM instance profile. We connect via a secure key-based SSH tunnel and install the complete system dependency stack (Nginx, PHP 8.3, and MariaDB):

```
ubuntu@ip-172-31-11-127: ~  
systemctl restart networkd-dispatcher.service  
systemctl restart serial-getty@ttyS0.service  
systemctl restart systemd-logind.service  
systemctl restart unattended-upgrades.service  
  
No containers need to be restarted.  
  
User sessions running outdated binaries:  
ubuntu @ session #5: sshd[2656,2753]  
ubuntu @ user manager service: systemd[2661]  
  
No VM guests are running outdated hypervisor (qemu) binaries on this host.  
ubuntu@ip-172-31-11-127:~$ sudo systemctl start nginx  
ubuntu@ip-172-31-11-127:~$ sudo sys  
syscount-bpfcc          systemd-creds           systemd-notify  
syscount.bt            systemd-cryptenroll     systemd-path  
sysctl                 systemd-cryptsetup      systemd-repart  
systemctl              systemd-delta           systemd-run  
systemd               systemd-detect-virt     systemd-socket-activate  
systemd-ac-power      systemd-escape          systemd-stdio-bridge  
systemd-analyze       systemd-firstboot       systemd-sysex  
systemd-ask-password  systemd-hwdb            systemd-sysusers  
systemd-cat           systemd-id128           systemd-tmpfiles  
systemd-cgls          systemd-inhibit         systemd-tty-ask-password-agent  
systemd-cgtop         systemd-machine-id-setup  
systemd-conftest      systemd-mount           systemd-umount  
ubuntu@ip-172-31-11-127:~$ sudo systemctl enable nginx  
Synchronizing state of nginx.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.  
Executing: /usr/lib/systemd/systemd-sysv-install enable nginx
```



```
ubuntu@ip-172-31-11-127: ~  
  
Normally, root should only be allowed to connect from 'localhost'. This  
ensures that someone cannot guess at the root password from the network.  
  
Disallow root login remotely? [Y/n] n  
... skipping.  
  
By default, MariaDB comes with a database named 'test' that anyone can  
access. This is also intended only for testing, and should be removed  
before moving into a production environment.  
  
Remove test database and access to it? [Y/n] y  
- Dropping test database...  
... Success!  
- Removing privileges on test database...  
... Success!  
  
Reloading the privilege tables will ensure that all changes made so far  
will take effect immediately.  
  
Reload privilege tables now? [Y/n] y  
... Success!  
  
Cleaning up...  
  
All done! If you've completed all of the above steps, your MariaDB  
installation should now be secure.  
  
Thanks for using MariaDB!  
ubuntu@ip-172-31-11-127:~$
```

```

ubuntu@ip-172-31-11-127: ~
All done! If you've completed all of the above steps, your MariaDB
installation should now be secure.

Thanks for using MariaDB!
ubuntu@ip-172-31-11-127:~$ sudo mysql -u root -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 40
Server version: 10.11.14-MariaDB-0ubuntu0.24.04.1 Ubuntu 24.04

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> CREATE DATABASE nextcloud CHARACTER SET utf8mb4 COLLATE utf8mb4_general_ci;
Query OK, 1 row affected (0.000 sec)

MariaDB [(none)]> CREATE USER 'nextcloud'@'localhost' IDENTIFIED BY
Query OK, 0 rows affected (0.002 sec)

MariaDB [(none)]> GRANT ALL PRIVILEGES ON nextcloud.* TO 'nextcloud'@'localhost';
Query OK, 0 rows affected (0.001 sec)

MariaDB [(none)]> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.001 sec)

MariaDB [(none)]> EXIT;
Bye
ubuntu@ip-172-31-11-127:~$

```

```

ubuntu@ip-172-31-11-127: /tmp
inflating: nextcloud/LICENSES/CC-BY-SA-4.0.txt
inflating: nextcloud/LICENSES/CC-BY-SA-3.0.txt
inflating: nextcloud/LICENSES/CC-BY-SA-3.0-IGO.txt
inflating: nextcloud/LICENSES/CC-BY-SA-2.0.txt
inflating: nextcloud/LICENSES/CC-BY-4.0.txt
inflating: nextcloud/LICENSES/CC-BY-2.0.txt
inflating: nextcloud/LICENSES/BSD-3-Clause.txt
inflating: nextcloud/LICENSES/BSD-2-Clause.txt
inflating: nextcloud/LICENSES/Apache-2.0.txt
inflating: nextcloud/LICENSES/AGPL-3.0-or-later.txt
inflating: nextcloud/LICENSES/AGPL-3.0-only.txt
inflating: nextcloud/COPYING
inflating: nextcloud/AUTHORS
inflating: nextcloud/.user.ini
inflating: nextcloud/.htaccess
ubuntu@ip-172-31-11-127: /tmp$ ls
latest.zip
nextcloud
snap-private-tmp
systemd-private-944e70b0fa5c456ebdf7100927170717-ModemManager.service-Px19a3
systemd-private-944e70b0fa5c456ebdf7100927170717-chrond.service-hAbCHk
systemd-private-944e70b0fa5c456ebdf7100927170717-fwupd.service-Io4SU7
systemd-private-944e70b0fa5c456ebdf7100927170717-polkit.service-ousY0B
systemd-private-944e70b0fa5c456ebdf7100927170717-systemd-logind.service-9etKJC
systemd-private-944e70b0fa5c456ebdf7100927170717-systemd-resolved.service-4Z78WZ
ubuntu@ip-172-31-11-127: /tmp$ sudo mv nextcloud /var/www/html/nextcloud
ubuntu@ip-172-31-11-127: /tmp$ sudo chown -R www-data:www-data /var/www/html/nextcloud
ubuntu@ip-172-31-11-127: /tmp$ sudo chmod -R 755 /var/www/html/nextcloud/
ubuntu@ip-172-31-11-127: /tmp$ sudo chmod -R 755 /var/www/html/nextcloud
ubuntu@ip-172-31-11-127: /tmp$

```

Downloaded and deployed Nextcloud core application files to the Nginx web root directory (/var/www/html/nextcloud) and assigned proper www-data ownership and file permissions.

```

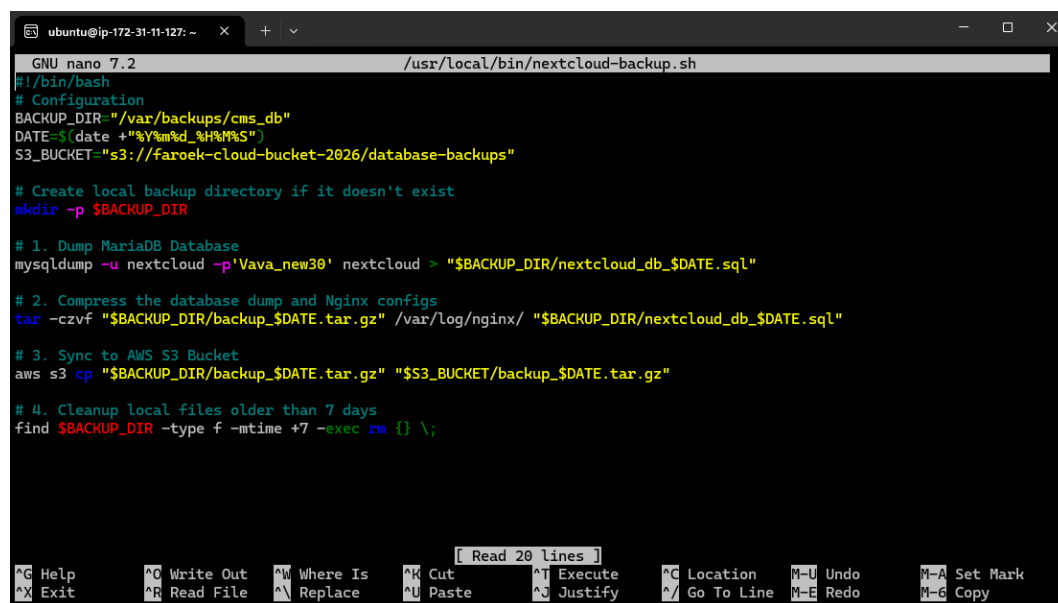
ubuntu@ip-172-31-11-127:~$ sudo nano /etc/nginx/sites-available/nextcloud
ubuntu@ip-172-31-11-127:~$ sudo ln -s /etc/nginx/sites-available/nextcloud /etc/nginx/sites-enabled/
ubuntu@ip-172-31-11-127:~$ sudo rm /etc/nginx/sites-available/default
ubuntu@ip-172-31-11-127:~$ sudo nginx -t
2026/09/07 06:50:04 [emerg] 2259#2259: open() "/etc/nginx/sites-enabled/default" failed (2: No such fi
le or directory) in /etc/nginx/nginx.conf:60
nginx: configuration file /etc/nginx/nginx.conf test failed
ubuntu@ip-172-31-11-127:~$ sudo rm /etc/nginx/sites-enabled/default
ubuntu@ip-172-31-11-127:~$ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
ubuntu@ip-172-31-11-127:~$ sudo sys
syscount-bpfcc      systemd-creds      systemd-notify
syscount.bt         systemd-cryptenroll systemd-path
sysctl              systemd-cryptsetup systemd-repart
systemctl            systemd-delta      systemd-run
systemd              systemd-detect-virt systemd-socket-activate
systemd-ac-power    systemd-escape      systemd-stdio-bridge
systemd-analyze      systemd-firstboot   systemd-sysext
systemd-ask-password systemd-hwdb         systemd-sysusers
systemd-cat          systemd-id128       systemd-tmpfiles
systemd-cgls         systemd-inhibit     systemd-tty-ask-password-agent
systemd-cgtop        systemd-machine-id-setup systemd-umount
systemd-confext      systemd-mount
ubuntu@ip-172-31-11-127:~$ sudo systemctl restart nginx
ubuntu@ip-172-31-11-127:~$

```

Resolved Nginx site-enabled symbolic link references and validated configuration syntax successfully.



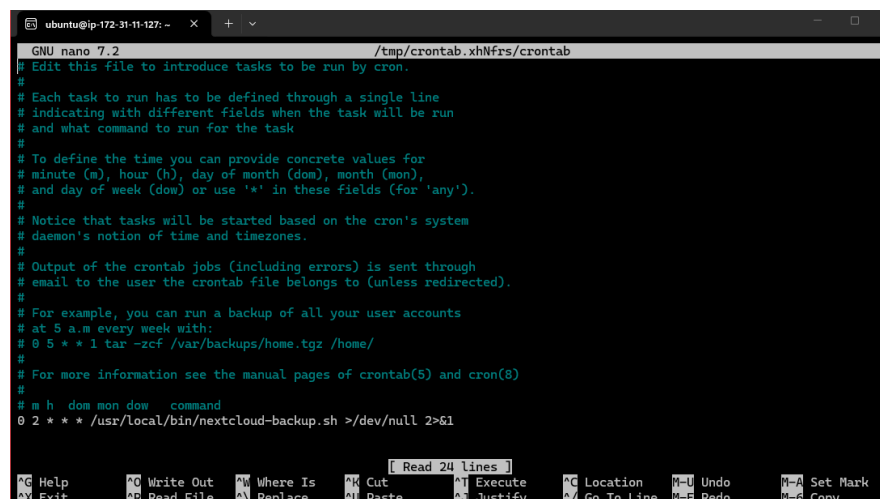
Created backup policy for s3

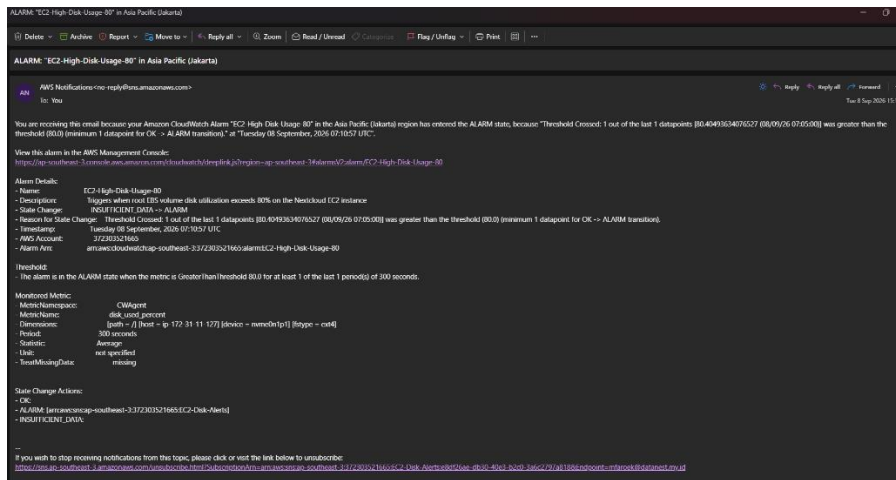


Automation for s3 backup script before cron

4. Automated Scripting & Cron Backup Scheduler

Disaster recovery is orchestrated by an automated Bash shell script that dumps the transactional database contents and web directories, bundles Nginx access/error logs, and uses the AWS CLI to sync archives into secure S3 partitions. We schedule this job to execute daily at 2:00 AM via Cron:





Cloudwatch alarm sent to email by sns aws services.

Testing & Operational Verification

Operational robustness was thoroughly verified across all system dimensions to guarantee the pipeline's.

readiness in a production-like environment:

- **Backup Pipeline Execution:** The Bash archival script was executed manually. We verified that database serialization dumps from `/var/backups/cms_db/` and compressed web directories successfully sync to the designated folders under the S3 bucket prefixes: `/database-backups/` and `/logs/nginx/`.
- **Log S3 Synchronization:** S3 bucket directory scans confirmed file creation with proper timestamps, zero transfer errors, and correct write encryption settings.
- **CloudWatch Telemetry Health:** Confirmed that memory and disk-percent utilization metrics are streamed every 60 seconds into the AWS Management Console.
- **SNS Alarm Dispatch:** Simulated disk capacity usage exceeding 80% on the EC2 instance volume. The CloudWatch threshold successfully fired, automatically triggering an AWS SNS topic alert and dispatching a detailed critical alert email.

Engineering & Operational Takeaways

Implementing this automated infrastructure provided valuable core engineering and systems administration

insights:

- **Linux Administration & Automation:** Developed hands-on competence in structured file ownership and security controls (www-data/chmod permissions), automation scheduler operations (cron syntax), LEMP service integrations, and shell automation script design.
- **Security Best Practices:** Reinforces how isolating security groups (restricting SSH ingress) and restricting AWS platform permissions via granular IAM policies protect critical backups against credentials leaks or node compromise.
- **Proactive Operational Resilience:** Highlighted the vital role of marrying disaster recovery processes (S3 backups) with real-time diagnostic performance metrics (CloudWatch) to preempt outages and secure high availability.